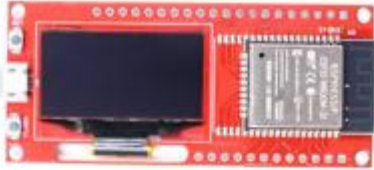


About this Lab

- What is the goal?
 - You are expected to read temperature, display it on screen, and upload it to the cloud
- How to do it?
 - We will first check the local functions are good
 - We will then upload the readings to AWS IoT Core

Devices for This Lab



MakePython

ESP32 x1



USB cable x1



DS18B20 module x1



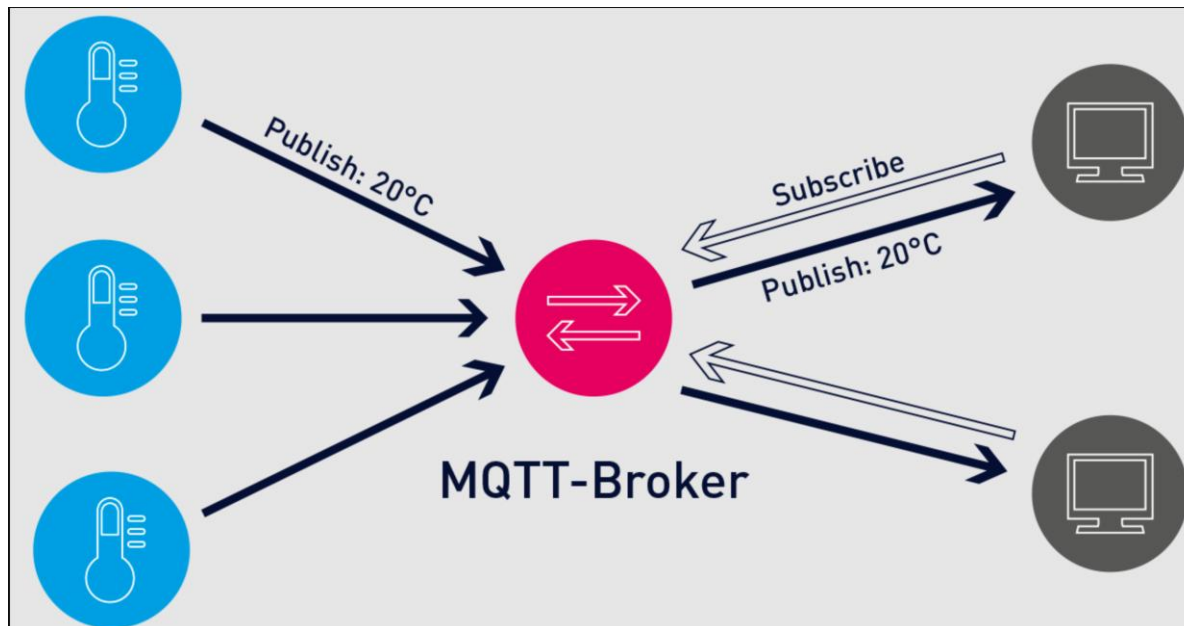
**Jump wire
(Female2Female) x3**

Background Knowledge

- **I2C** protocol to control the OLED screen
 - You have learnt about it in class
- **1-wire** protocol to read from temperature sensor
 - [Overview of 1-Wire Technology and Its Use | Analog Devices](#)
- SUTD-Wifi offers enterprise enhanced security by **PEAP**
 - Different from WPA(2) in ordinary home applications
 - You have also found it harder to connect :(
 - Google/GPT it if you're interested in, this is more about wireless security instead of this course's concerns
- **MQTT** protocol to send message to AWS IOT in internet
 - A brief here and more to learn next week

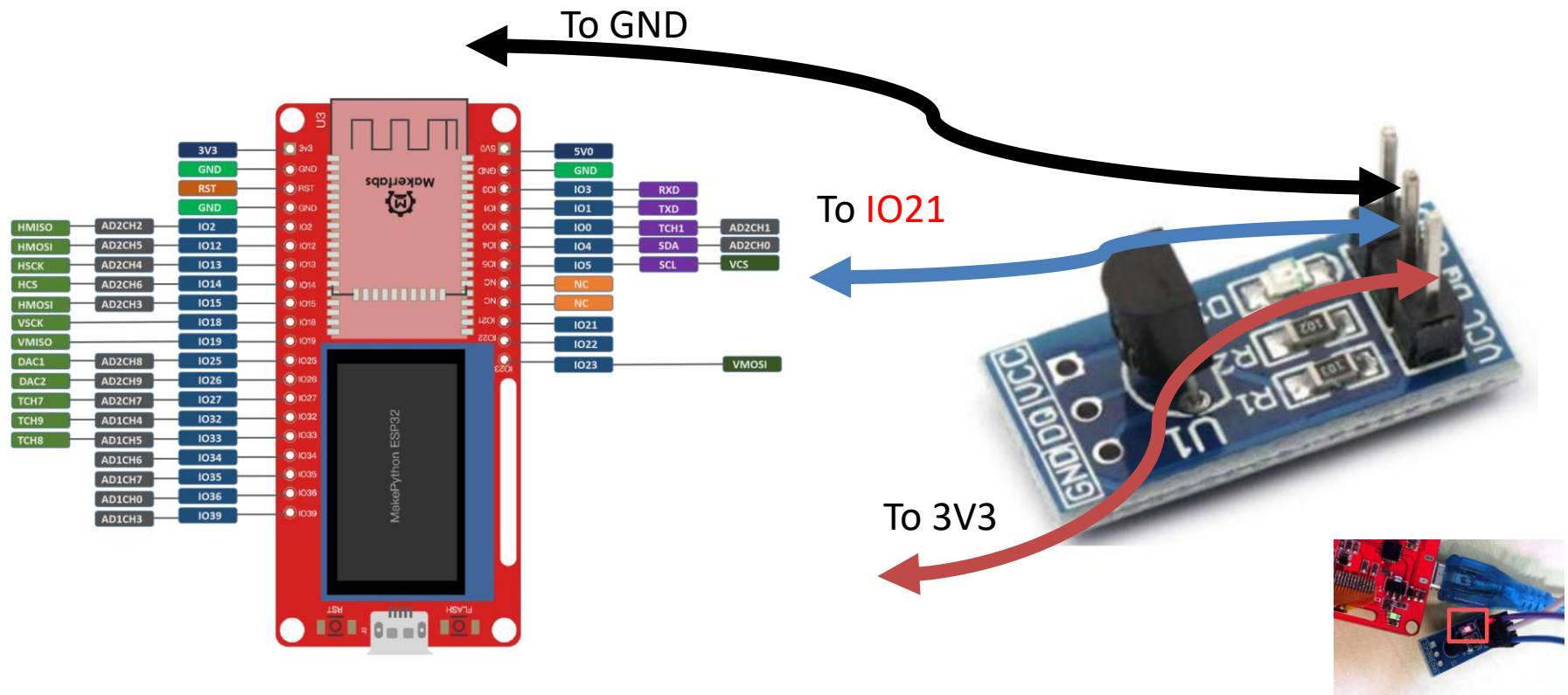
MQTT protocol

- What MQTT (Message Queuing Telemetry Transport)?
 - Lightweight messaging protocol
 - Publish/Subscribe model
 - Designed for low-bandwidth, high-latency, unreliable networks
- Core components:
 - Publisher, subscriber, broker
- Use cases:
 - IoT Devices
 - Real-time Analytics
 - Mobile Applications



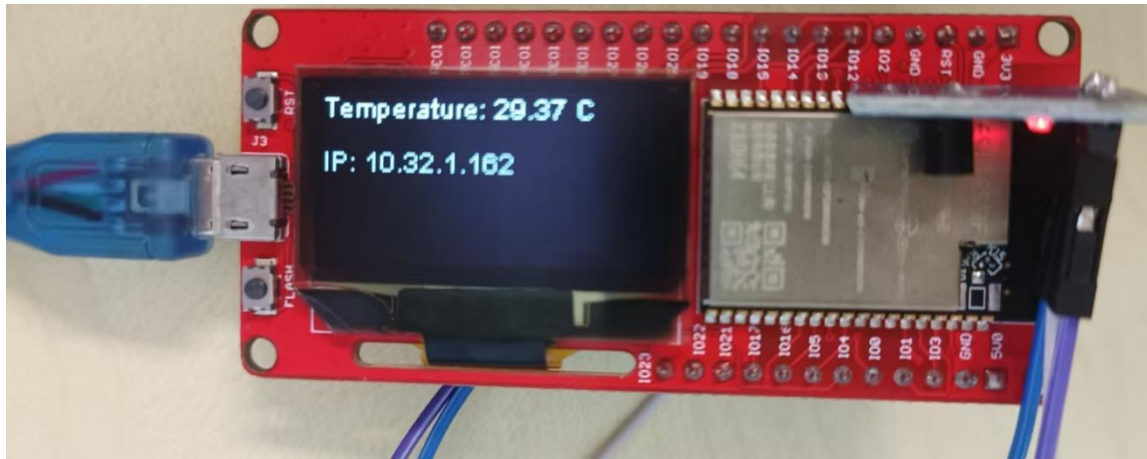
Part 1: Local Functions Test

- Connect the DS18B20 (temperature sensor)
 - Only 'D0' carries information to IO21, i.e., **1-wire!**
 - Others are power supply



Part1 :Local Functions Test

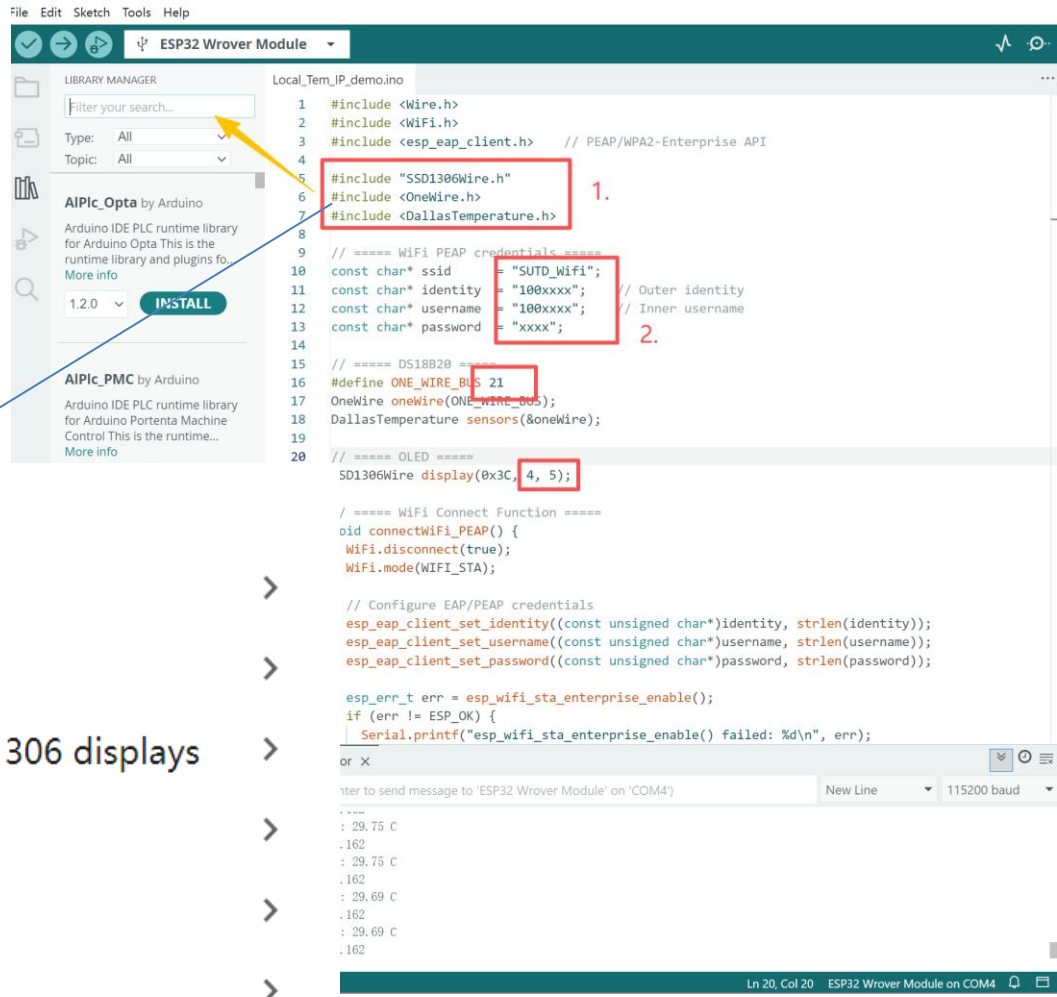
- The expected result
 - The screen has words to display
 - You can connect to SUTD-Wifi and get the IP
 - The temperature readings are displayed



- Reference code:

Arduino IDE: https://github.com/nan2li/SUTD_CCIOT_AWS-IOT-core_ESP32_PCAP-wifi/tree/main

ESP-IDF IDE: <https://github.com/TONYSKYZENG/CCIOTLabs>



ArduinoJson

DallasTemperature

ESP8266 and ESP32 OLED driver for SSD1306 displays

MQTT

OneWire

PubSubClient

- Reference code: https://github.com/nan2li/SUTD_CCIOT_AWS-IOT-core_ESP32_PCAP-wifi/tree/main

Part 1: Local Functions- ESP-IDF

- Get the code by

```
git clone https://github.com/TONYSKYZENG/CCIOTLabs.git  
-b lab1_2_1
```

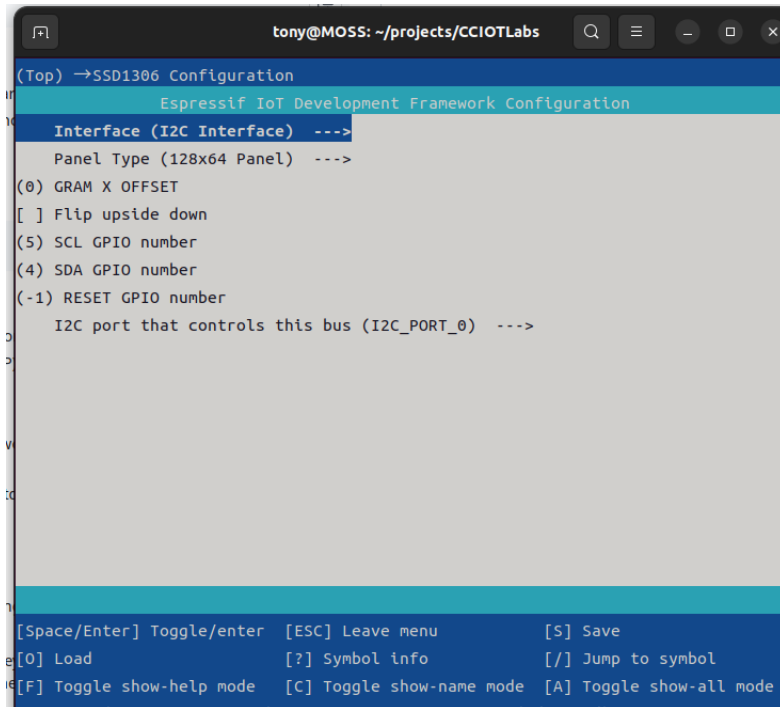
- Enter the CCIOTLabs folder and see the README.md
- Recall hints if you can not find the serial port
 - Run `idf.py flash` or `idf.py monitor` directly without specifying the port
- Recall hints if you can not evoke idf.py in ubuntu/mac:
 - Do the following before running idf.py

```
export IDF_PATH=~/.esp/esp-idf  
source $IDF_PATH/export.sh
```

- Reference code: <https://github.com/TONYSKYZENG/CCIOTLabs>

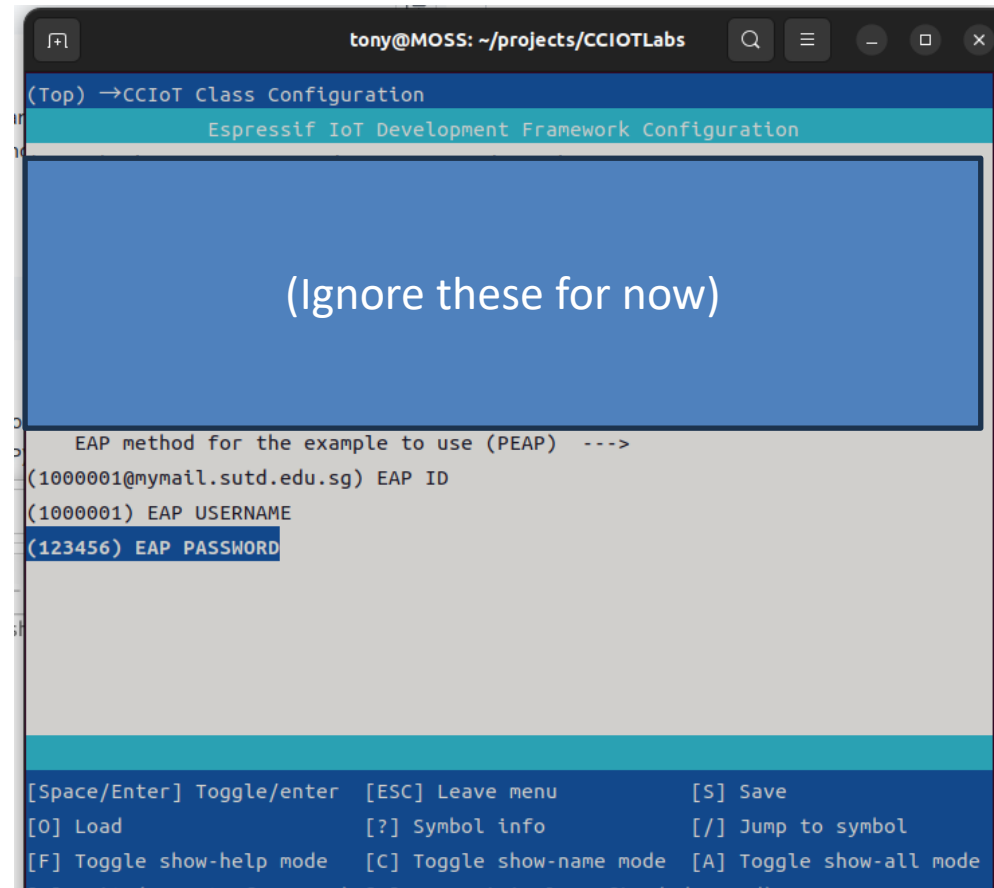
Part 1: Local Functions- ESP-IDF

- Configure SUTD-Wifi*



```
tony@MOSS: ~/projects/CCIOTLabs
(Top) →SSD1306 Configuration
Espressif IoT Development Framework Configuration
Interface (I2C Interface) --->
  Panel Type (128x64 Panel) --->
  (0) GRAM X OFFSET
  [ ] Flip upside down
  (5) SCL GPIO number
  (4) SDA GPIO number
  (-1) RESET GPIO number
  I2C port that controls this bus (I2C_PORT_0) --->

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                    [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode [A] Toggle show-all mode
```



```
tony@MOSS: ~/projects/CCIOTLabs
(Top) →CCIOT Class Configuration
Espressif IoT Development Framework Configuration

(Ignore these for now)

EAP method for the example to use (PEAP) --->
(1000001@mymail.sutd.edu.sg) EAP ID
(1000001) EAP USERNAME
(123456) EAP PASSWORD

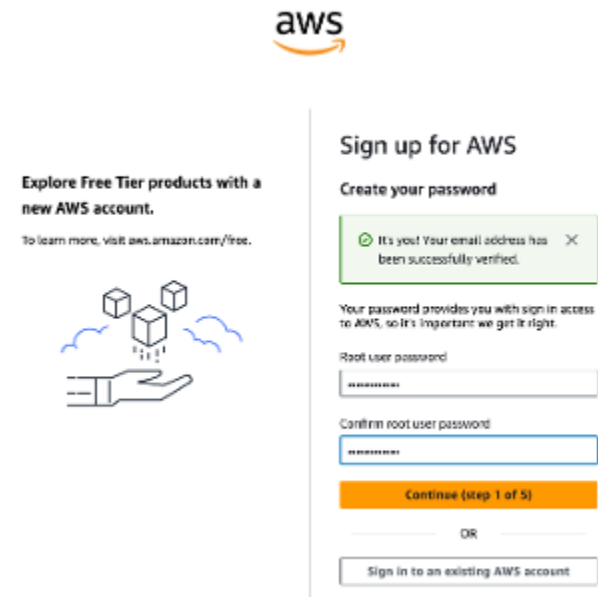
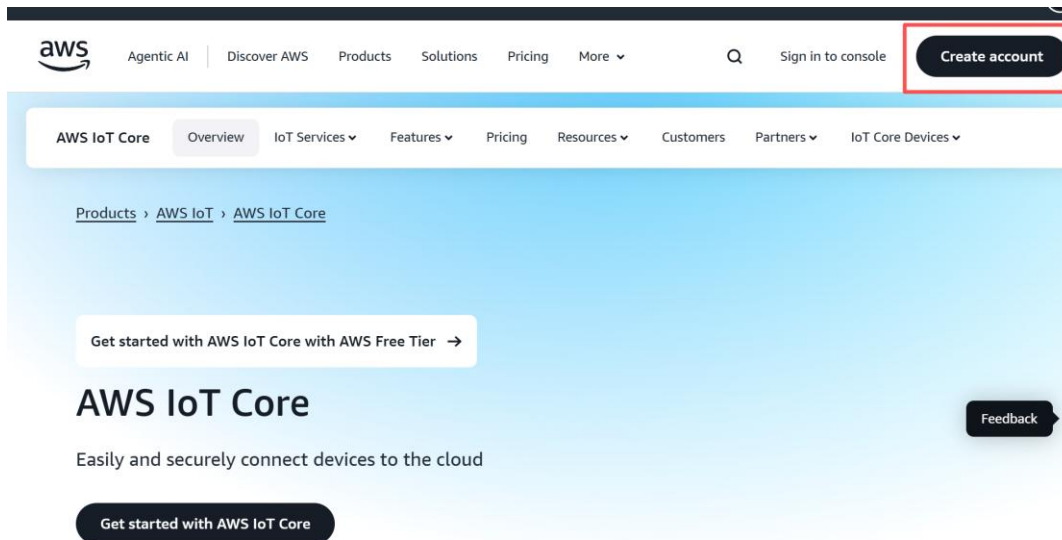
[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                    [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode [A] Toggle show-all mode
```

* See also the example provided by ESP for more details of PEAP:

https://github.com/espressif/esp-idf/tree/release/v4.3/examples/wifi/wpa2_enterprise

Part2: Configuring AWS

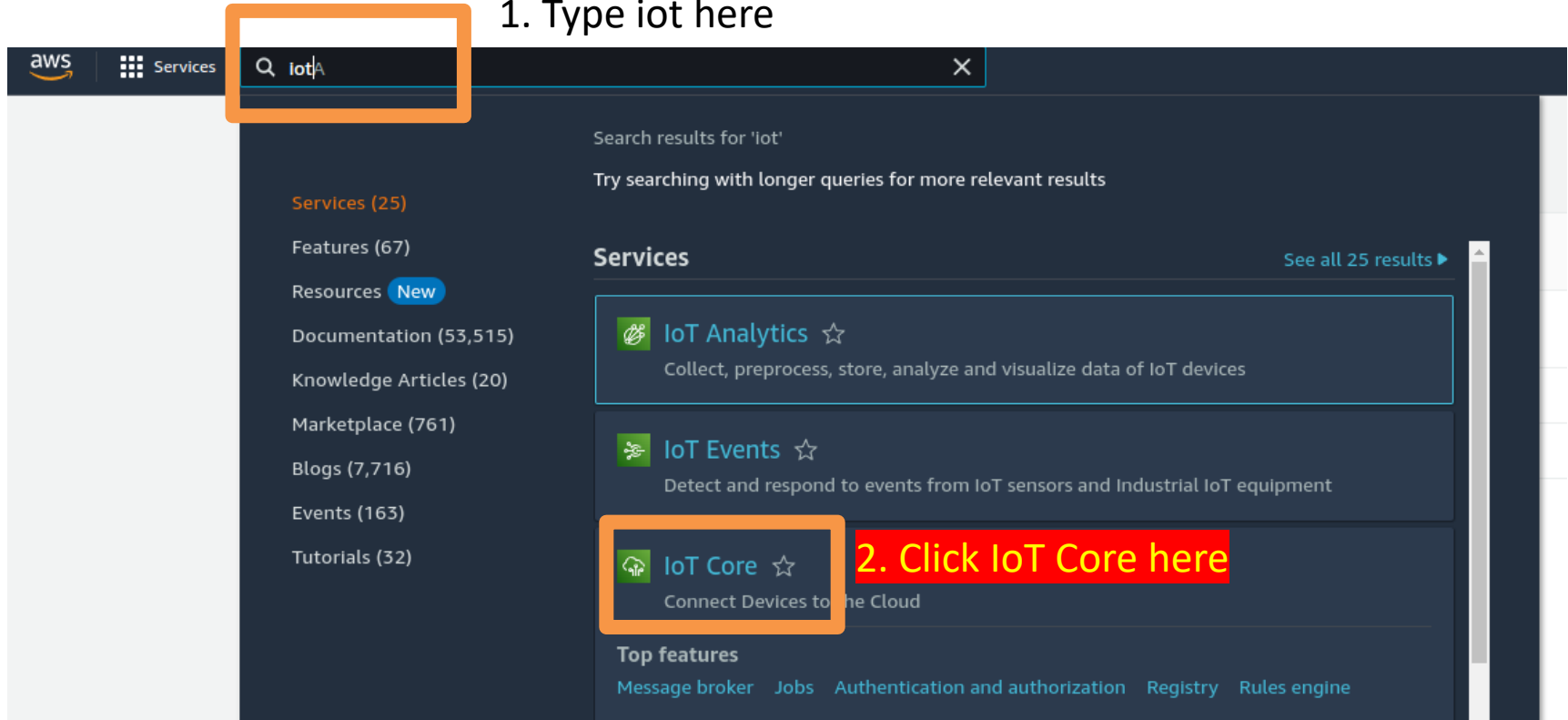
- Find the AWS login page at <https://aws.amazon.com/iot-core/>
- Register first, if you don't have an account.



Part2: Configuring AWS

- Find the IoT core after log in

1. Type iot here



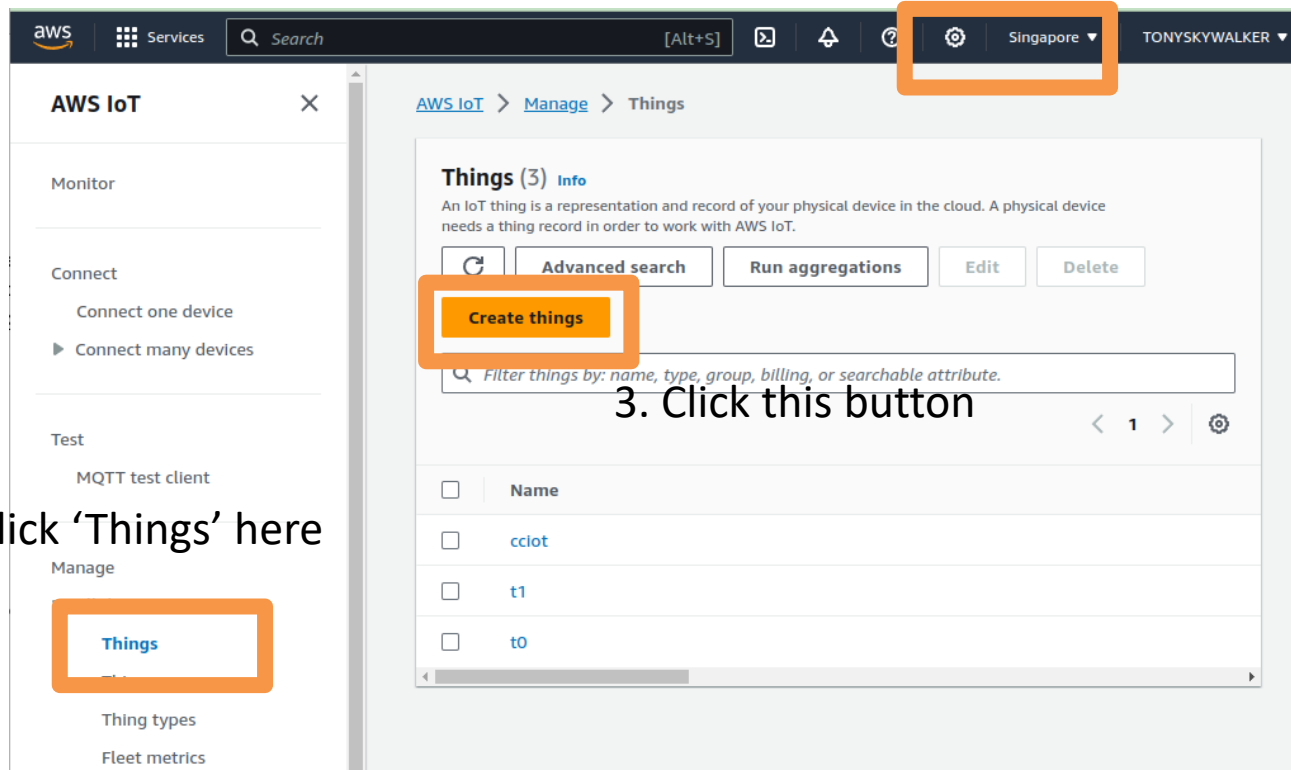
Part2: Configuring AWS

- Create an AWS IoT device *

1. Select Singapore here

2. Click 'Things' here

3. Click this button



Ref: <https://esp32io.com/tutorials/esp32-aws-iot>

<https://how2electronics.com/connecting-esp32-to-amazon-aws-iot-core-using-mqtt/>

Part3: Connect AWS and ESP32

1. Create an AWS IoT device to get the CA related information.
2. Identify all necessary parameters for connecting the IoT device to AWS Cloud.

Ref: <https://esp32io.com/tutorials/esp32-aws-iot>

<https://how2electronics.com/connecting-esp32-to-amazon-aws-iot-core-using-mqtt/>

```
images.h secrets.h SSD1306SimpleDemo.ino
1 #include <pgmspace.h>
2
3 #define SECRET
4 #define THINGNAME "ESP32-temp" 1
5
6 // WiFi (WPA2-Enterprise PEAP)
7 #define WIFI_SSID "SUTD_Wifi"
8 #define WIFI_IDENTITY "1" 2
9 #define WIFI_USERNAME "" 2
10 #define WIFI_PASSWORD "" 2
11 const char AWS_IOT_ENDPOINT[] = "a1n6v8qkxwzpk1-ats.iot.ap-southeast-1.amazonaws.com" 3
12
13
14 // Amazon Root CA 1
15 static const char AWS_CERT_CA[] PROGMEM = R"EOF( 4
16 -----BEGIN CERTIFICATE-----
17 MIIDQTCGAimgAwIBAgITBmyfz5m/jAo54vB4ikPmljZbyJANBgkqhkiG9w0BAQSF
18 ADA5MQswCQYDVQQGEwJVUzEPMA0GA1UEChMGQWVudGVudC51dWVudC51dWVudC51
19 b24gUm9vdCBMRzEuaXN0eXBvc3R1b3R1b3R1b3R1b3R1b3R1b3R1b3R1b3R1b3R1
20 MA0GA1UdDgQWBABMBG8wKQYDVR0tAQH/MAsGCWcwggEwMBIGA1UdEAAQQAQDA
ca9HgFB0fw7Y14h29...Rvo3bSMgHFzZM
cmzU5L/qw
whmahRWa6
di1L
IIEgeW...
CME WYDVR0TAQH/...
AYYwHQYDVRR0...B3DQEBCwIA
A4IBAQC8jd...G/wkjlhuQ/KMS
USPMCCji...T53iHT+103f...msv0tadQ1wUs
Ma...aACbvXy8Mky7Vu33PqUXHeeF6V/U...96LXFvkWlJ3Yk8U90vv
...gZK1b1uZWgJcJmApzyMZFO6IQ6XU
5MsI+yMRQ+hDKXJioaldXgjUkK642M4UwtBV8ob2xJNDd2ZhWLnOqeXeGADbkpy
rqXRfboQnoZsG4q5WTP468SQvvG5
-----END CERTIFICATE-----
)EOF";
37
38 // Device Certificate
39 static const char AWS_CERT_CRT[] PROGMEM = R"KEY(
40 -----BEGIN CERTIFICATE-----
41 MII...CAAKGgAwIBAgIUQBawDSUjqATQXi...RY4MkJmYsQwDQYJKoZIhvcNAQEL
42 ...TTE...CA1UdEAAQQAQDA...
```

Part3: Connect AWS and ESP32

- Reminder*

Download certificates and keys

Download certificate and key files to install on your device so that it can connect to AWS.

Download certificates and keys

Download certificate and key files to install on your device so that it can connect to AWS.

Device certificate
9111e51108b...te.pem.crt

Deactivate certificate

Download

Certificate downloaded

Key files

The key files are unique to this certificate and can't be downloaded after you leave this page. Download them now and save them in a secure place.

This is the only time you can download the key files for this certificate.

Public key file
9111e51108b26bfae2e8304...ef452cf-public.pem.key

Download

Key downloaded

Private key file
9111e51108b26bfae2e8304...f452cf-private.pem.key

Download

Key downloaded

Root CA certificates

Download the root CA certificate file that corresponds to the type of data endpoint and cipher suite you're using. You can also download the root CA certificates later.

Amazon trust services endpoint
RSA 2048 bit key: Amazon Root CA 1

Download

Certificate downloaded

Amazon trust services endpoint
ECC 256 bit key: Amazon Root CA 3

Download

Certificate downloaded

If you don't see the root CA certificate that you need here, AWS IoT supports additional root CA certificates. These root CA certificates and others are available in our developer guides. [Learn more](#)

Download all Done

AWS IoT

Monitor

Connect

Test

Manage

Connect one device

Connect many devices

Domain configurations

MQTT test client

Query connectivity status

Thing groups

Thing types

Fleet metrics

Things (3)

Advanced search

Run aggregations

Run connectivity status

Create things

An IoT thing is a representation and record of your physical device in the cloud. A physical device needs a thing record in order to work with AWS IoT.

Filter things by: name, type, group, billing, or searchable attribute.

Name
ESP32-temp
ESP32
Makepython-esp32_temperature

AWS IoT

Monitor

Connect

Test

Manage

Connect one device

Connect many devices

Domain configurations

MQTT test client

Query connectivity status

Thing groups

Thing types

Fleet metrics

Domain configurations

A custom endpoint that devices use to connect to AWS IoT. Use different endpoints for different fleets to specify different authentication methods and protocols.

How it works

Step 1. Select authentication types and protocols

AWS provides various authentication methods and protocols to accommodate different security requirements and device types. For instance, a smart printer might connect using X.509 client certificates with secure MQTT, while a smartphone may utilize SigV4 over HTTPS.

Step 2. Create a new domain configuration to configure devices

Each domain configuration specifies the authentication methods, protocols, and provides a dedicated endpoint for device connections. Configure devices to connect to the endpoint and include the domain name as the Server Indication (SNI) during the TLS handshake.

Find domain configurations

Name	Domain name
iot:Data-ATS	a16yrtu4upw81r-ats.iot.ap-southeast-1.amazonaws.com

Part3: Connect AWS and ESP32

- Reminder*

The screenshot shows the Arduino IDE interface. On the left, the 'LIBRARY MANAGER' is open, displaying the 'AIPLc_Opta' library by Arduino. A yellow arrow points from the 'LIBRARY MANAGER' tab to the 'AIPLc_Opta' library entry. The main editor shows the 'SSD1306SimpleDemo.ino' file. The code includes various headers and defines for an ESP32 IoT project. Red boxes and numbers highlight specific parts of the code:

- Red box 1: `#include <PubSubClient.h>`, `#include <ArduinoJson.h>`, `#include <OneWire.h>`, and `#include <DallasTemperature.h>` (lines 8-11).
- Red box 2: `#define ONE_WIRE_BUS 21` (line 14).
- Red box 3: `"esp32/esp32-to-aws/pub"` (line 15) and `"esp32/aws-to-esp32/sub"` (line 16).

The code in the background is as follows:

```
1 /*
2  * ESP32 AWS IoT with DS18B20 Temperature Sensor
3  * Updated for real DS18B20 module
4  */
5
6 #include "secrets.h"
7 #include <WiFiClientSecure.h>
8 #include <MQTTClient.h>
9 #include <ArduinoJson.h>
10 #include "WiFi.h"
11 #include <OneWire.h>
12 #include <DallasTemperature.h>
13
14 // DS18B20 data pin
15 #define ONE_WIRE_BUS 21
16
17 // The MQTT topics
18 #define AWS_IOT_PUBLISH_TOPIC "esp32/esp32-to-aws"
19 #define AWS_IOT_SUBSCRIBE_TOPIC "esp32/aws-to-esp32"
20
21 #define PUBLISH_INTERVAL 4000 // 4 seconds
22
23 WiFiClientSecure net = WiFiClientSecure();
24 MQTTClient client = MQTTClient(256);
25
26 // Setup a oneWire instance to communicate with any OneWire
27 OneWire oneWire(ONE_WIRE_BUS);
28
29 // Pass our oneWire reference to Dallas Temperature library
30 DallasTemperature sensors(&oneWire);
31
32 unsigned long lastPublishTime = 0;
33
34 void setup() {
35   Serial.begin(115200);
36 }
```

Part3: Connect AWS and ESP32

- The intended effect:

Connect

- Connect one device
- Connect many devices
- Domain configurations

Test

- MQTT test client**
- Query connectivity status

Manage

- All devices
 - Things
 - Thing groups
 - Thing types
 - Fleet metrics
- Greengrass devices
- Software packages
- Remote actions
- Message routing
- Retained messages
- Security
- Fleet Hub

Subscribe to a topic Publish to a topic

Topic filter Info

The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include wildcards.

esp32/esp32-to-aws/pub

Additional configuration

Subscribe

Subscriptions

esp32/pub ❤️ ✕

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

Additional configuration

Publish

▼ esp32/pub

```
{
  "device": "ESP32-temp",
  "temperature_celsius": 27.6875,
  "temperature_fahrenheit": 81.8375,
  "timestamp": 315118
}
```

Connect

- Connect one device
- Connect many devices
- Domain configurations

Test

- MQTT test client**
- Query connectivity status

Manage

- All devices
 - Things
 - Thing groups
 - Thing types
 - Fleet metrics

To disconnect from the MQTT test client, choose Disconnect. To re-establish a connection, choose Connect.

Subscribe to a topic **Publish to a topic**

Topic name

The topic name identifies the message. The message payload will be published to this topic with a QoS of 0.

Q esp32/aws-to-esp32/sub ✕

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

Additional configuration

Publish



```
Published:
{"device": "ESP32-temp", "temperature_celsius": 27.625, "temperature_fahrenheit": 81.725, "timestamp": 315118}
Incoming on [esp32/sub]
{
  "message": "Hello from AWS IoT console"
}
Parsed message: Hello from AWS IoT console
```